

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION INFORMATICĂ ENGLEZĂ

DIPLOMA THESIS

Doclane

Supervisor
Lect. PETRESCU Manuela

Author
Robert Béres

2026

ABSTRACT

Doclane is a web application that helps citizens interact with public institutions without needing to visit in person multiple times. In many countries, including Romania, people still need to go to offices, wait in lines, and bring printed documents. This takes a lot of time and effort.

The problem is that most current systems are not fully digital, or they only cover part of the process. Some systems allow document submission but not tracking or feedback. Others work only for one institution.

Doclane tries to fix this by letting users submit requests online, upload documents, and see the status of their application. Employees at the institution can review the documents and send feedback if something is wrong or missing. This way, users do not need to visit in person just to find out that a document they provided is missing something.

The system is built as a web application using Go for the backend and Next.js for the frontend. It is fully cloud-native and scalable, adhering to modern best practices.

The thesis also looks at existing systems like GOV.UK, Enter Finland, and DocuSign. These systems solve parts of the problem but not all of it. Doclane tries to bring these ideas together in one system that can work across different institutions.

Doclane does not replace the physical pickup of final documents, since some documents require this by law. But it removes most of the unnecessary visits that happen before that step, which is where most of the time and frustration comes from.

Contents

1	Introduction	1
1.1	Introducing the problem	1
1.2	Role of Information Technology in Public Administration	1
1.3	Challenges in Traditional Systems	2
1.4	Limitations of existing digital solutions	3
1.5	Proposed solution	3
1.6	Objectives	3
1.7	Scope of the system	4
2	Related work	5
2.1	Automated electronic document management systems in municipal units	5
2.2	Digital document systems in public sector implementation	6
2.3	Enter Finland	6
2.4	DocuSign	6
2.5	GOV.UK Services	7
2.6	Comparison	7
2.7	Gap	7
3	Implementation	9
3.1	Backend	9
3.1.1	Go	9
3.1.2	Layers	9
3.1.3	Authentication	10
3.1.4	API Response	10
3.1.5	Custom Errors	11
3.1.6	Configuration	11
3.1.7	Middleware	11
3.1.8	File Storage	11
3.1.9	Routing	11
3.2	Database	12

3.3	Frontend	13
4	Deployment	14
4.1	Cloud computing	14
4.2	Amazon Web Services	14
4.2.1	Containers and Kubernetes	15
4.3	The architecture	15
4.3.1	Networking	15
4.3.2	Database layer	16
4.3.3	Backend layer	17
4.3.4	Frontend layer	17
4.3.5	Ingress and Load Balancing	18
4.3.6	DNS and HTTPS	18
4.3.7	Document storage	18
4.3.8	Authentication	18
4.3.9	Logging and monitoring	19
4.4	Machine Learning	19
4.4.1	Textract	19
4.4.2	Polly	19
4.4.3	Bedrock	19
4.5	Infrastructure as Code - Terraform	19
4.5.1	State management	20
4.5.2	Stack layout	20
4.5.3	Ephemeral environments	21
4.6	Continuous Integration / Continuous Deployment	21
4.7	Containerization	22
4.7.1	Backend container	22
4.7.2	Frontend container	22
5	User workflow	23
5.1	Registration and login	23
5.2	Submitting a request	23
5.3	Review and feedback process	23
5.4	Administrator workflow	24
5.5	Department workflow	24
5.6	AI-assisted document processing	25
6	System limitations and discussion	26
6.1	System scope	26
6.2	Dependence on external services	26

6.3	AI limitations	27
6.4	User adoption challenges	27
6.5	Scalability and future use	27
6.6	Summary	27
7	Conclusion	28
7.1	Future improvements	28
	Bibliography	30

Chapter 1

Introduction

1.1 Introducing the problem

Public institutions play an important role in the daily lives of citizens, providing access to services such as official documents, permits, and administrative procedures. Because of this, it is important that the processes involved are efficient, reliable, and easy to use.

In many cases, these processes are still based on physical documents and in-person interactions. Citizens are often required to visit institutions, wait in queues, and submit printed files in order to complete simple requests. This approach leads to long waiting times, repeated visits, and inefficient handling of information.

In addition, physical documents can be lost, damaged, or incorrectly processed, which can delay important procedures. These inefficiencies do not only create inconvenience, but can also impact the ability of citizens to complete time-sensitive administrative tasks.

As a result, improving the way public institutions handle document-based workflows becomes an important problem to address.

1.2 Role of Information Technology in Public Administration

The use of information technology in public institutions has become increasingly important in recent years. Digital systems allow institutions to manage large amounts of data, automate processes, and reduce the time required to complete administrative tasks.

By moving services to digital platforms, interactions between citizens and institutions can be simplified. Instead of relying on physical documents and in-person

visits, many processes can be handled online, improving accessibility and reducing waiting times.

In addition, digital systems make it easier to store, organize, and retrieve information. This can improve decision-making and reduce errors caused by manual processing of documents.

However, the adoption of such systems is not always straightforward. Public institutions may face challenges such as limited financial resources, lack of technical infrastructure, and insufficient digital skills among both employees and citizens.

Despite these challenges, the use of information technology remains a key factor in improving the efficiency and quality of public services.

1.3 Challenges in Traditional Systems

Traditional document handling in public institutions comes with several practical challenges that affect both citizens and employees.

One of the main issues is the need for physical presence. In most cases, citizens are required to visit an institution in person in order to submit or collect documents. This creates queues and waiting times, especially during busy hours or peak periods.

Another problem is the use of paper-based documents. Files are printed, copied, and stored physically. This takes space and also creates the risk of documents being lost, damaged, or misplaced. It also makes it harder to search and manage information when needed.

The process of checking and validating documents is often manual. Employees need to go through each file one by one, which takes time and can lead to delays, especially when the number of requests is high.

Feedback to citizens is also not always immediate. If a document is missing or incorrect, the citizen usually finds out later, sometimes after waiting for a long time or making another visit. This leads to repeated submissions and additional waiting time.

In addition, many of these processes are not transparent to the user. Citizens often do not know the current status of their request or how long it will take until it is completed.

These challenges show that the traditional way of handling document-based workflows is slow and can be improved with more structured and digital solutions.

1.4 Limitations of existing digital solutions

Even though some public services already use digital platforms, they are often limited in scope. Most of these systems are built for a specific type of service or a single institution.

In many cases, users still need to switch between different systems depending on the service they need. This creates fragmentation and makes the overall process less consistent.

Some systems also only cover part of the workflow, such as submitting documents or signing them, but not the full process from submission to final approval and feedback.

Because of this, there is still a need for systems that can be reused across different institutions, where each institution can use its own instance of the same workflow structure instead of relying on completely separate tools.

1.5 Proposed solution

The proposed system, Doclane, is a web-based application designed to handle document-based workflows between citizens and public institutions.

The system allows users to submit requests, upload required documents, and track the status of their applications. On the other side, institution employees can review submissions, validate documents, and provide feedback when needed.

The goal of the system is to reduce the need for physical visits and repeated submissions by moving the process into a structured digital workflow.

Doclane also focuses on making the process more clear for users by showing what is required at each step and giving feedback when something is missing or incorrect.

1.6 Objectives

The main objective of this thesis is to design and implement a system that improves document-based workflows in public institutions.

The specific objectives include:

- Designing a backend system that can handle document requests.
- Implementing secure authentication and role-based access.
- Building a structured workflow for submission and validation.
- Using cloud storage for document handling.

- Making the system scalable and easy to extend.

1.7 Scope of the system

The system focuses on improving the workflow between citizens and public institutions when it comes to document-based requests.

It does not change the legal process of document approval. Final decisions are still made by authorized personnel within the institution.

The system is designed as a support tool that helps organize and digitalize the existing workflow, not replace institutional rules or procedures.

Chapter 2

Related work

This chapter presents existing systems and research related to digital document handling and public administration workflows. The goal is to show how similar problems have been handled in other systems and what solutions already exist.

The systems and studies presented here focus on document submission, electronic workflows, and online public services. Some of them are real systems used by governments, while others are research papers that propose ways to improve administrative processes.

After presenting these systems, a short comparison is made to identify their limitations in relation to the problem addressed by this thesis. This helps explain the motivation behind Doclane and how it differs from existing solutions.

2.1 Automated electronic document management systems in municipal units

Koptyakova et al. [KZM19] present a system for automated electronic document management in municipal institutions. The paper focuses on improving how public institutions handle documents and reducing the time needed for administrative work.

The authors explain that traditional document workflows in municipal units are slow and depend on manual work. Documents are usually registered, tracked, and processed by hand, which can cause delays when the number of requests is high.

To solve this, the paper proposes an automated system that digitizes document registration, tracking, and processing inside municipal institutions. This reduces manual work and makes the process faster.

The study also looks at the cost and benefits of such systems. It mentions that the payback period can be short because less manual work is needed and efficiency increases.

However, the system is mainly used inside one municipal institution. It focuses on internal document handling and not on the full process between citizens and institutions.

Compared to this, Doclane focuses on the full workflow between citizens and public institutions, including submission, validation, and feedback.

2.2 Digital document systems in public sector implementation

Az Ab Aziz et al. [AYMJ19] study the implementation of a Digital Document Management System (DDMS) in the Malaysian public sector. The system is used to manage documents and records inside government institutions and aims to reduce paper use and improve document handling.

The paper shows that even when such systems exist, adoption is not always high. The main issues are not only technical, but also related to policy, training, security, and internal support.

The authors explain that successful implementation depends on clear guidelines and coordination between agencies. Without this, different institutions use the system in different ways, which reduces its effectiveness.

Compared to this, Doclane focuses more on workflows between citizens and institutions, but the same idea applies: a system is only useful if it is properly adopted and used in real work.

2.3 Enter Finland

Enter Finland (<https://enterfinland.fi>) is a government service used in Finland for immigration applications. Users can fill forms online, upload documents, and pay fees. After submission, some cases still require an in-person visit.

The system reduces the need for paper submission and physical visits at the start of the process. However, it is limited to immigration services and one type of institution.

Compared to this, Doclane is designed for different types of document workflows across multiple institutions.

2.4 DocuSign

DocuSign (<https://www.docusign.com>) is a platform used for signing documents online. Users can upload documents, send them for signature, and track

their status.

It removes the need to print and scan documents in many cases. It is mostly used in business environments.

However, it focuses mainly on signing and approval. It does not support full administrative workflows like structured requests or multi-step validation.

Compared to this, Doclane handles full workflows from submission to approval and feedback.

2.5 GOV.UK Services

GOV.UK (<https://www.gov.uk>) is a UK government platform for online public services. Users can apply for documents, submit forms, and access different services.

It reduces the need for physical visits and allows many tasks to be done online. It covers many services in one platform.

However, each service has its own workflow and is not part of one unified system.

Compared to this, Doclane aims to provide a single workflow system for different types of document requests.

2.6 Comparison

The systems described above solve parts of the same problem in different ways.

Enter Finland is focused on one domain. GOV.UK provides many services, but they are specifically for one institution alone. DocuSign focuses on document signing.

All of them reduce paper use and improve digital access, but they are limited either by domain, structure, or function.

2.7 Gap

The systems described above solve parts of the problem but none of them cover the full workflow in a way that works across different institutions. They are either built for one domain, one country, or one specific function like signing.

Outside of these platforms, many institutions rely on software that is built specifically for them. This kind of tailored software is expensive and takes time to develop. For smaller public institutions with limited budgets, this is often not a realistic option.

Doclane tries to fill this gap by offering a simple, general system that can be used across different institutions without needing custom development. The goal is not to replace large government platforms, but to give smaller institutions a working solution that covers the full document workflow from submission to feedback.

Chapter 3

Implementation

The backend of Doclane is written in Go, and the frontend is built with Next.js. Go was chosen because it is fast, has a large standard library, and handles errors in a way that makes bugs easier to track. Next.js was chosen because Doclane is a dynamic application that makes many requests to the backend, and server-side rendering helps keep loading times short.



📁 .github/workflows	feat: granular security works	last month
📁 aws	feat: gitignore update	1 minute ago
📁 backend	feat: gitignore update	1 minute ago
📁 docs	new cloud architecture (using github actions instead of aws...	last month
📁 frontend	feat: almost done	last week
📄 .gitignore	feat: gitignore update	1 minute ago
📄 LICENSE	Update LICENSE	4 months ago
📄 README.md	feat: readme	last month
📄 init.sql	feat: gitignore update	1 minute ago

Figure 3.1: Overview of Doclane’s folder structure.

3.1 Backend

3.1.1 Go

Go is a statically typed, compiled programming language. It has a simple syntax and a standard library that covers most common needs. It is also fast to write and produces efficient binaries.

3.1.2 Layers

The backend code is split into four layers:

- **Handler layer:** handles incoming HTTP requests and extracts the needed information from them, such as JWT claims, resource IDs, or uploaded files.
- **Service layer:** contains the business logic and validations. For example, it checks whether a user has the right role to perform an action. A user cannot view the files of a request unless they are an admin or they belong to the department that manages that request.
- **Repository layer:** executes queries on the database. It trusts everything passed to it, since all validation happens in the service layer. Transactions are used when multiple database operations need to succeed together for an action to be considered complete.
- **Model layer:** holds the data structures used across the application, such as users, requests, templates, invitation codes, and comments. DTOs (Data Transfer Objects) are used in some cases to enrich data from the database. For example, when retrieving a request, the DTO also includes the requester's name and email, not just their ID.

3.1.3 Authentication

Authentication is handled using JWTs (JSON Web Tokens) issued by AWS Cognito. A JWT is a stateless token that contains information about the user such as their ID, role, department, and name. The information inside the token is readable by anyone, but it cannot be changed without the signature becoming invalid. The backend rejects any token with an invalid signature.

The token is stored in an HTTP-only cookie. This means JavaScript running in the browser cannot access it, which protects against XSS (Cross-Site Scripting) attacks. More details about Cognito and how it is configured are in chapter 4.

3.1.4 API Response

Every response from the API follows the same structure. This makes it easier for the frontend to handle responses in a consistent way. Every response contains:

- **success** - a boolean that says whether the request worked or not.
- **message** - optional, a short description of what happened.
- **error** - optional, an error message present when success is false.
- **data** - optional, contains any data returned by the request.


```
2
3  type APIResponse struct {
4      Success bool `json:"success"`
5      Data    any   `json:"data,omitempty"`
6      Msg     string `json:"message,omitempty"`
7      Err     string `json:"error,omitempty"`
8  }
9
```

Figure 3.2: API Response struct.

3.1.5 Custom Errors

Custom error types are defined and used across the backend. This gives a unified way to handle errors and makes the code easier to read.

3.1.6 Configuration

There is a single configuration file that acts as the main setup for the application. It is where all service and repository instances are created and exported. If a repository implementation needs to be changed, only this file needs to be updated. Everything else continues to work without changes.

3.1.7 Middleware

Most routes in the application are protected by middleware. The middleware checks that every incoming request has a valid JWT token before it reaches the handler. Requests with missing or invalid tokens are rejected early.

There are two middleware checks in Doclane. The first checks that the token is valid and was signed correctly by Cognito. The second checks that the user account is active.

3.1.8 File Storage

Uploaded files are stored in S3, which is a file storage service from AWS. The file storage logic is separated from the rest of the application so the app does not depend on it directly. All file access is done through presigned URLs, which are temporary links that give access to a specific file for a limited time. More details about S3 and how it is set up are in chapter 4.

3.1.9 Routing

Backend routing is done using the Chi library, which is a third party Go package. Although Go's standard library handles many things well, HTTP routing is an area

where external help is useful. Chi makes it easy to organize routes in a clear and readable way.

```

64     r.Get("/api/invitations/info", invitation_handler.GetInvitationCodeInfoHandler)
65     r.Post("/api/internal/process-recurring", request_handler.ProcessRecurringRequestsHandler)
66
67     r.Route("/api", func(r chi.Router) {
68         r.Use(auth_middleware.AuthGuard)
69         r.Use(auth_middleware.MustBeActive)
70
71         r.Get("/stats", stats_handler.GetStatsHandler)
72
73         r.Route("/users", func(r chi.Router) {
74             r.Get("/", user_handler.GetUsersHandler)
75             r.Get("/me", user_handler.GetCurrentUserHandler)
76             r.Patch("/me/profile", user_handler.UpdateProfileHandler)
77             r.Patch("/me/password", user_handler.UpdatePasswordHandler)
78             r.Get("/by-department", user_handler.GetUsersByDepartmentHandler)
79             r.Get("/{id}", user_handler.GetUserByIDHandler)
80             r.Post("/notify/{id}", user_handler.NotifyUserHandler)
81             r.Post("/deactivate/{id}", user_handler.DeactivateUserHandler)
82             r.Patch("/{id}/department", user_handler.UpdateUserDepartmentHandler)
83         })
84
85         r.Route("/requests", func(r chi.Router) {
86             r.Get("/", request_handler.GetAllRequestsHandler)
87             r.Post("/", request_handler.AddRequestHandler)
88             r.Get("/assignee/{id}", request_handler.GetRequestsByAssigneeHandler)
89             r.Get("/department/{id}", request_handler.GetRequestsByDepartmentHandler)

```

Figure 3.3: Routing for the backend.

3.2 Database

PostgreSQL was chosen as the database engine. It is one of the most widely used relational databases and it fits the needs of Doclane well since the data has clear relationships between tables.

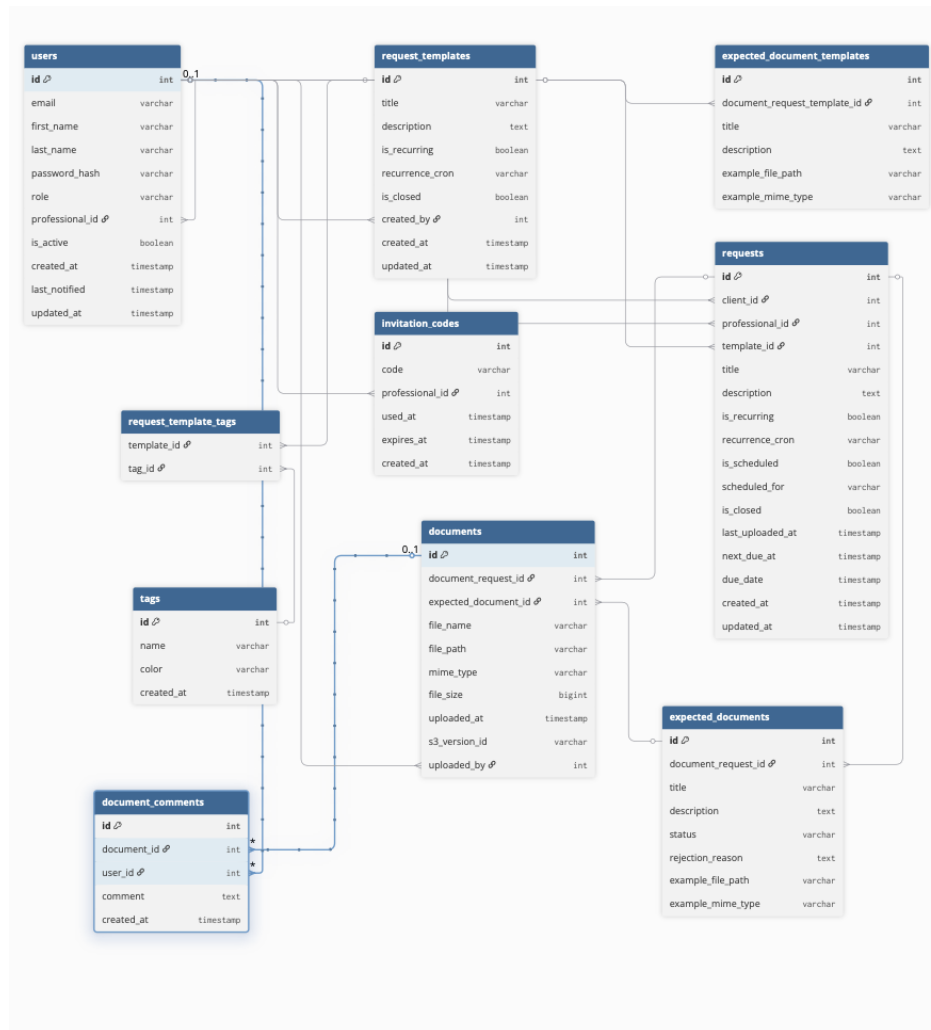


Figure 3.4: Database schema overview.

3.3 Frontend

The frontend is built with Next.js, which is an open-source web framework built on top of React. It supports server-side rendering, which means pages can be prepared on the server before being sent to the browser.

Next.js was chosen because Doclane makes many requests to the backend on almost every page. If a client-side only framework like plain React was used, users would spend a lot of time on loading screens. With Next.js, data is fetched on the server before the page is shown, so the experience is faster.

Frontend code is split into small reusable components so that the same piece of UI does not need to be written more than once. This follows the DRY (Don't Repeat Yourself) principle, which is also applied in the backend but is most visible in the frontend.

Chapter 4

Deployment

When it comes to deploying an application, there are many paths one could take. One might be ordering a physical server at home and running your application on there, another one could be running it locally on your machine where only people in your network can access it, and another one could be deploying the application in the cloud. The last one was chosen for Doclane.

4.1 Cloud computing

Cloud computing was preferred since it is the more modern approach, and reduces overhead with managing physical servers. Without the burden and expenses of buying, setting up and maintaining servers, more effort could be redirected to the development of the app and adding features that bring more value to it. It is also that renting those servers means you can just get rid of them whenever you're not using them anymore, and you only pay for how much you use them.

4.2 Amazon Web Services

The deployment of the app was done on AWS. The architecture leverages containers orchestrated by Kubernetes, specifically the managed EKS (Elastic Kubernetes Service) offering from AWS.

AWS is the leading cloud provider, covering roughly 31% of the global market as of 2026. Part of the reason it is so widely used is because it was the first to market, so it had the most amount of time out of all competitors to refine their services and offerings, which is half of the reason it was chosen. The other half was because the author of this thesis project is a certified Solutions Architect Professional on AWS and had a bias to pick this provider.

4.2.1 Containers and Kubernetes

A container is a lightweight, standalone package that includes everything needed to run a piece of software: the code, runtime, libraries and dependencies. Unlike virtual machines, containers share the host operating system kernel, making them fast to start and efficient with resources.

Docker is the tool used to build these containers. The backend and frontend of Doclane each have a Dockerfile that defines how the container image is built. The backend produces a small static Go binary running on a minimal base image, while the frontend uses Next.js standalone output on a Node.js base image.

Kubernetes is an open-source container orchestration platform. It takes your containers and runs them across a cluster of machines, handling scaling, networking, health checks and restarts automatically. Instead of manually starting containers on servers, you declare what you want (for example: "run 2 copies of the backend") and Kubernetes makes it happen.

EKS (Elastic Kubernetes Service) is AWS' managed Kubernetes offering. AWS handles the control plane (the brain of the cluster), while you manage the worker nodes (the machines that actually run your containers). This removes the operational burden of maintaining Kubernetes itself while giving full access to its features.

4.3 The architecture

The architecture consists of an EKS cluster running in a VPC with two availability zones. Traffic enters through an ALB (Application Load Balancer) that routes requests to the appropriate service: the backend or the frontend. The backend connects to an RDS PostgreSQL database and an S3 bucket for document storage. Authentication is handled by Cognito.

4.3.1 Networking

The networking part of Doclane consists of a VPC (Virtual Private Cloud) in the eu-west-1 Region on AWS. This VPC consists of two private subnets and two public subnets. The private subnets host the EKS worker nodes and the RDS database, so that no direct public internet access can be established to them.

The public subnets host the NAT Gateway and the ALB (Application Load Balancer). The NAT Gateway allows resources in private subnets to reach the internet (for example to pull container images) without being directly accessible from the internet.

Both the private and public subnets span two AZs (Availability Zones), such that high availability is achieved. In case one data center from AWS goes down where our application runs, then it will still continue to run as if nothing has happened.

An internet gateway was provisioned and added to the route table of the public subnet, such that internet access is possible to and from it. Private subnets are associated with a route table that routes internet-bound traffic through the NAT Gateway. So the thing that makes a subnet public or private is whether or not the associated route table has a direct route to the internet gateway or goes through a NAT Gateway.

The subnets are tagged with Kubernetes-specific labels so that the AWS Load Balancer Controller can automatically discover which subnets to place ALBs in.

Security

When it comes to network security, the architecture leverages Security Groups for securing traffic at the pod level and at the RDS level.

A Security Group in AWS is a stateful firewall that operates at a single resource level. It consists of inbound / outbound rules, inbound rules meaning what kind of traffic is allowed (on which port and from which source), while outbound rules control what traffic is allowed to go out of our resource.

The RDS database is protected by a Security Group that only allows inbound traffic on port 5432 from the EKS cluster Security Group. This means only pods running in the cluster can reach the database. Any other traffic is dropped.

The entire communication between the backend pods and RDS happens over encrypted connections (SSL/TLS is enforced on the database). Traffic between pods within the cluster uses the internal Kubernetes network and never leaves the VPC.

4.3.2 Database layer

RDS (Relational Database Service) was the preferred choice for the database. As a relational database was needed, the natural choice was AWS' managed service for this.

RDS is built in such a way that AWS handles all the operating system patching of the server that is running your database, and you also benefit from automated backups out of the box.

The engine is PostgreSQL, chosen for being popular, fast and well supported by RDS. The database schema is automatically applied on every fresh deployment by a Kubernetes Job that runs the initialization SQL script.

4.3.3 Backend layer

The backend runs as a Deployment in Kubernetes with 2 replicas by default. Each replica is a container running the Go binary that serves the chi HTTP router on port 8080.

A HorizontalPodAutoscaler is configured to scale the backend from 2 to 6 replicas based on CPU utilization. If average CPU goes above 70%, Kubernetes automatically adds more pods to handle the load.

Health checks are configured using Kubernetes liveness and readiness probes. The liveness probe ensures crashed containers get restarted. The readiness probe ensures traffic is only sent to pods that are ready to serve requests.

IAM Roles for Service Accounts (IRSA)

The backend pods need access to AWS services like S3, Textract, Bedrock, Polly and Cognito. Instead of using static credentials, the pods use IRSA (IAM Roles for Service Accounts).

IRSA works by linking a Kubernetes Service Account to an IAM Role. When a pod uses that Service Account, the AWS SDK automatically receives short-lived credentials scoped to exactly the permissions defined in the IAM Role. No static keys exist anywhere, not in the image, not in environment variables, not on disk.

The IAM policy attached to the backend pod role follows the principle of least privilege. It only allows the specific actions needed: uploading and downloading from the documents S3 bucket, calling Textract for text extraction, calling Bedrock for document interpretation, calling Polly for speech synthesis, and looking up users in Cognito.

4.3.4 Frontend layer

The frontend runs as a Deployment in Kubernetes with 2 replicas. Each replica runs the Next.js standalone server on port 3000. The standalone output mode produces a minimal server bundle that does not need the full `node_modules` directory, keeping the container image small.

A HorizontalPodAutoscaler scales the frontend from 2 to 4 replicas based on CPU utilization.

Server-side rendering (SSR) calls from the frontend to the backend happen over the internal Kubernetes network using the service DNS name. This means SSR requests never leave the cluster and have minimal latency.

4.3.5 Ingress and Load Balancing

Traffic enters the cluster through an ALB (Application Load Balancer) that is automatically provisioned by the AWS Load Balancer Controller. This controller runs inside the cluster and watches for Ingress objects. When it sees one, it creates and configures a real ALB in AWS to match the routing rules defined in the Ingress.

The Ingress defines two routing rules: requests to `/api` are sent to the backend service, and all other requests are sent to the frontend service. The ALB terminates TLS using an ACM certificate and redirects HTTP traffic to HTTPS.

The ALB uses IP-mode target groups, meaning it routes traffic directly to pod IPs rather than going through node ports. This is more efficient and allows for faster health checks.

4.3.6 DNS and HTTPS

The application is served at `thesis.robert-beres.com`. An ACM (AWS Certificate Manager) certificate was provisioned for this domain and validated automatically via DNS. The certificate is attached to the ALB so all traffic is encrypted.

A Route53 alias record points the domain to the ALB. When the infrastructure is torn down and recreated, the ALB gets a new DNS name, but Terraform automatically updates the Route53 record to point to the new one.

4.3.7 Document storage

S3 (Simple Storage Service) is used for storing uploaded documents. The bucket has versioning enabled so previous versions of files are preserved, and server-side encryption is applied to all objects.

The backend generates presigned URLs for downloads. This means clients download directly from S3 using a temporary URL that expires after a set time, without any permanent credentials ever existing. This makes the process so much more secure since if somehow credentials get leaked the attack surface is very small due to the scoped permissions, and the attack surface is also ephemeral.

4.3.8 Authentication

Cognito is used as the identity provider. It manages user registration, login, password policies and token issuance. The frontend uses the Amplify library to interact with Cognito directly from the browser.

After a user logs in, Cognito issues a JWT (JSON Web Token). This token is stored in an HTTP-only cookie and sent with every request. The backend validates

the token by checking its signature against Cognito's public keys (JWKS).

4.3.9 Logging and monitoring

Container logs from all pods are collected by Kubernetes and can be viewed using `kubecttl`. The structured JSON logging from the backend makes it easy to filter and search for specific events or errors.

4.4 Machine Learning

This section goes into detail on the AWS services leveraged to build the machine learning assisted features aforementioned in the implementation chapter.

4.4.1 Textract

Textract is a fully managed serverless service offered by AWS that extracts text from documents. API calls are made to Textract from Doclane in order to extract text from documents.

4.4.2 Polly

Polly is a fully managed serverless service offered by AWS that generates speech from text. API calls are made to Polly to generate speech from selected documents.

4.4.3 Bedrock

Bedrock is a fully managed serverless service offered by AWS that gives you access to different LLMs that you can prompt. In our case we use the Nova Lite model from Amazon to interpret whether a document provided by an user is actually what it is required by the request.

4.5 Infrastructure as Code - Terraform

Whenever deploying applications in the Cloud, you can manually click in the AWS console and provision everything you need. It's straight forward, intuitive and it provides a friendly experience. Although this is convenient, it is a manual process. And manual processes are prone to errors. It becomes easy to forget what change in configuration you made last week, more simply put, it's a mess.

That's where the concept of IaC (Infrastructure as Code) comes into play. This involves specifying your infrastructure in a declarative manner, and having everything defined as code, it becomes much easier to maintain and have control over.

Terraform by HashiCorp is the chosen tool for the project. Unlike CloudFormation which is AWS-specific, Terraform is cloud-agnostic and has a large ecosystem of providers. It was chosen for its ergonomics: features like `force_destroy` on S3 buckets, native Helm and Kubernetes providers, cross-region resource management with provider aliases, and the ability to manage the entire stack (AWS resources, Kubernetes manifests, Helm charts) from a single tool.

4.5.1 State management

Terraform keeps track of what it manages in a state file. This file maps the resources defined in code to the real resources in AWS. It is stored remotely in an S3 bucket with versioning enabled, so it can be recovered if corrupted. A DynamoDB table is used as a lock to prevent two people (or two CI runs) from modifying infrastructure at the same time.

4.5.2 Stack layout

The infrastructure is split into multiple Terraform stacks, each with its own state file:

bootstrap - Creates the S3 bucket and DynamoDB table for state management. Run once, never destroyed.

platform-data - Contains long-lived resources that are cheap to keep running: ECR container registries, Cognito user pool, ACM certificate, GitHub OIDC role. These persist across compute teardowns.

platform-compute - Contains the expensive resources: VPC, NAT Gateway, EKS cluster, RDS database, S3 documents bucket, IRSA roles. These are created when working and destroyed when not needed.

workload - Contains everything inside the Kubernetes cluster: Helm charts, deployments, services, ingress, database seeding jobs. Destroyed together with platform-compute.

This split allows tearing down the expensive compute resources while not using the app, since in development you do not need the app running 24 hours a day, while keeping the cheap persistent resources alive. A full environment can be recreated from scratch with a single command.

4.5.3 Ephemeral environments

The entire compute layer is designed to be ephemeral. Running `make up` provisions the full environment from scratch in about 15 minutes: VPC, NAT, EKS cluster, RDS database, container deployments, database schema seeding, admin user creation, ALB, HTTPS, DNS record. Running `make down` destroys everything in about 10 minutes, bringing the cost to zero.

This is possible because:

- Container images persist in ECR between sessions
- The Cognito user pool persists so users do not need to re-register
- The ACM certificate persists so DNS validation does not need to happen again
- The database schema is applied automatically by a Kubernetes Job on every fresh deployment
- The admin user is bootstrapped automatically by another Kubernetes Job

4.6 Continuous Integration / Continuous Deployment

Modern practices of deployment have been adopted, such that the deployment pipeline of the application is fully automated. Once changes are pushed to the main branch, container images are automatically built and pushed to ECR.

CI/CD has been done with the help of GitHub Actions. GitHub Actions are convenient to use when your repository is already on GitHub, hence why they were chosen.

The pipeline builds Docker images for the backend and frontend, tags them with the git commit SHA for traceability, and pushes them to ECR. The backend and frontend are built independently, only when their respective code changes.

Updates happen leveraging the least-privilege principle, without any permanent credentials ever existing or being stored anywhere. The GitHub Action worker generates an ID token using GitHub's OIDC (OpenID Connect). GitHub signs this token with their private key, and this signed token is then passed to AWS to prove the identity of the runner.

In AWS, an OIDC provider is configured in IAM, trusting GitHub and allowing Role Assumptions for the worker as long as it proves it's from the Doctlane repository. The IAM role assumed by the worker only has permissions to push images to the two ECR repositories. Nothing else.

Role Assumption in AWS is a concept that allows generating temporary credentials to perform actions in an AWS account, leveraging the STS (Secure Token Service) from AWS. Credentials are short-lived. Once the credentials expire they cannot be used anymore, and even if they somehow get leaked the damage would be minimal since the only thing they allow is pushing container images.

4.7 Containerization

Both the backend and frontend are packaged as Docker containers using multi-stage builds.

4.7.1 Backend container

The backend Dockerfile uses a two-stage build. The first stage uses the Go compiler to build a fully static binary with no external dependencies. The second stage uses a minimal distroless base image that contains no shell, no package manager, nothing except the binary and CA certificates. The final image is about 36 MB and runs as a non-root user.

4.7.2 Frontend container

The frontend Dockerfile also uses a multi-stage build. The first stage installs dependencies and runs the Next.js build. The second stage copies only the standalone output (a minimal server bundle) into a Node.js Alpine image. The final image is about 370 MB and runs as a non-root user.

The NEXT_PUBLIC environment variables (Cognito pool ID, client ID, backend URL) are passed as build arguments since Next.js inlines them into the JavaScript bundle at build time. They are not secrets, they are public values that the browser needs to know.

Chapter 5

User workflow

This chapter describes how the system is used by end users and explains typical workflows for common scenarios in daily use.

5.1 Registration and login

The first step for using the system is creating an account. The user must register by providing basic information such as name, email, and password. After registration, the account is created and can be used to access the system.

After registration, the user can log in using their email and password. Once logged in, the system creates a session for the user and allows access to the available features based on their role.

5.2 Submitting a request

After logging in, the user needs to complete their profile by adding personal information such as address and phone number.

After the profile is completed, the user can submit a request. The user can choose a request template from the available list and fill in the required information. Depending on the template, the user may also need to upload supporting documents.

Once the request is submitted, it is sent to the relevant department in the public institution. Employees review the submitted information and check whether all required documents are correct and complete.

5.3 Review and feedback process

During the review process, each submitted document is checked individually by the department employee. Each document can be either approved or rejected sepa-

rately.

If a document is correct and meets the requirements, it is marked as approved. If a document is incorrect, missing information, or does not meet the requirements, it is rejected.

When a document is rejected, the employee can add a reason for the rejection. This helps the user understand what needs to be corrected or resubmitted. The user receives this feedback directly in the system.

After receiving feedback, the user can update the request by uploading corrected documents. The request is then reviewed again until all documents are approved.

When all documents in a request are approved, the request is marked as completed and the user is notified. The user can then collect the final document from the institution if needed.

5.4 Administrator workflow

Administrators are responsible for managing request templates in the system.

An administrator can create a new template for a specific type of request. Each template defines what documents are required and what information the user must provide. The administrator can also add example files for each required document to help users understand what is expected.

In addition, templates can be organized using tags. Tags help group similar templates together and make it easier to search and manage them inside the system.

5.5 Department workflow

Each request belongs to a specific department inside the institution. Employees from that department are responsible for handling incoming requests.

When a new request is submitted, it becomes available to department members. A department member can then claim a request, which means they become responsible for reviewing and processing it.

After claiming a request, other employees can see that it is already assigned and avoid duplicate work. The assigned employee continues the review process until the request is completed.

This system ensures that each request has a clear owner and improves organization inside the department.

5.6 AI-assisted document processing

The system includes AI-based tools to help with document processing. These tools are used to assist employees during the review of uploaded documents, especially when the document is unclear or hard to read.

One of the main use cases is when a document is blurry or of low quality. In this case, an AI service can analyse the image and estimate whether the document matches the expected type defined in the request template. This helps the employee decide faster if the document is valid or needs to be rejected.

Another feature is text extraction from images. If a document is scanned or uploaded as a photo, an OCR (Optical Character Recognition) service can extract the text content from it. This makes it easier to read and search through documents without manually inspecting every file.

In some cases, a text-to-speech (TTS) service can also be used to convert document content into audio. This can help users or employees who prefer listening instead of reading, or when reviewing long documents.

These AI tools do not make final decisions. They only provide suggestions or extracted information, while the final approval or rejection is always done by the assigned department employee.

Chapter 6

System limitations and discussion

This chapter discusses the limitations of the current system and explains some design decisions made during development. While the system covers the main workflow of document processing in public institutions, there are still areas that can be improved.

6.1 System scope

Doclane is designed as a workflow system for handling document-based requests between users and public institutions. It does not change legal procedures or replace existing administrative rules. Final decisions are still made by authorized employees within the institution.

The system focuses only on digital support of existing processes. It does not automate decision-making in a legal sense, but instead helps organize and speed up document handling.

6.2 Dependence on external services

The system uses external services for file storage and AI-based processing. This includes cloud storage for uploaded files and third-party services for tasks such as text extraction and document analysis.

This introduces a dependency on external systems. If one of these services is unavailable, some features of the system may be limited. However, the core functionality of the system remains usable.

6.3 AI limitations

AI tools are used to assist with document processing, but they do not make final decisions. They can extract text from images or help identify document types, but the final validation is always done by the assigned employee.

In some cases, AI results may not be fully accurate. Because of this, AI is only used as a support tool and not as a replacement for human review.

6.4 User adoption challenges

One possible limitation of the system is user adoption. Some users may not be familiar with digital systems or may prefer traditional paper-based processes.

In addition, employees in public institutions may require time to adapt to the new workflow. Training and experience are important for the system to be used effectively.

6.5 Scalability and future use

The system is designed to be scalable and support multiple institutions. However, large-scale adoption would require additional configuration, integration with national systems, and stronger legal frameworks.

At the current stage, the system serves as a prototype that demonstrates how document workflows can be improved using digital tools.

6.6 Summary

The system successfully covers the main workflow of document submission and processing, but it still has limitations related to external dependencies, AI accuracy, and user adoption. These limitations define the areas for future improvement and further development of the system.

Chapter 7

Conclusion

This thesis presented Doclane, a web-based system for managing document-based workflows between citizens and public institutions. The system was designed to reduce the need for physical visits and paper-based processes by moving the workflow into a digital environment.

The main goal of the system is to make the submission and processing of documents simpler and more structured. Users can submit requests online, upload required documents, and track the status of their application. Employees in public institutions can review submissions, approve or reject documents, and provide feedback when needed.

The system also introduces role-based workflows where each request is handled by a responsible employee. This helps keep the process organized and avoids confusion in document handling.

In addition, AI-based tools were integrated to assist in document processing. These tools can help extract text from images, check document quality, and provide additional support during the review process. However, final decisions are always made by the assigned employee.

The system was implemented using a Go backend and a Next.js frontend. Cloud services were used for file storage and deployment, making the system scalable and accessible.

Although the system covers the full workflow between users and institutions, there are still areas that can be improved in the future.

7.1 Future improvements

A key improvement would be the integration of eIDAS-compliant digital signing. This would allow users to verify their identity in a stronger way and reduce or fully remove the need for physical presence at public institutions.

Another improvement would be scheduling in-person visits within the app, such that another burden is taken away from the public institutions Doclane is trying to help. So as there are requests right now, once all documents are approved, the employee in the public institution would invite the citizen to schedule a meeting given some time constraints he has, from which a user would pick what is convenient to him.

These improvements would make the system more secure, more transparent, and closer to a fully digital public administration workflow.

Bibliography

- [AYMJ19] Azlina Ab Aziz, Zawiyah Mohammad Yusof, Umi Asma Mokhtar, and Dian Indrayani Jambari. The implementation guidelines of digital document management system for malaysia public sector: Expert review. *Journal of Physics: Conference Series*, 1333:072034, 2019.
- [KZM19] S V Koptyakova, E G Zinovyeva, and T V Maiorova. Development and deployment of automated electronic document management system in municipal units. *Journal of Physics: Conference Series*, 1333, 2019.